

Complete Feature Manifest

Exhaustive inventory of every module, class, function, API endpoint, algorithm, integration, test suite, and subsystem in the ReconPro platform.

96 Python Source Files | 31 Test Files | 1,449 Tests | 85+ Classes | 200+ Functions

156 Files Total | 121,000+ Lines of Code | Version 11.0.0

Enterprise Reconnaissance Platform

Pure Python | Zero Required Dependencies | Production-Ready

Table of Contents

1. Executive Summary & Grand Totals
2. Architecture Overview
3. Core Infrastructure (8 modules)
4. Scanning Engine & Orchestration (5 modules)
5. Network & Protocol Layer (6 modules)
6. Reconnaissance Modules - Standard (10 modules)
7. Advanced Reconnaissance Modules (16 modules)
8. Intelligence Systems (7 modules)
9. Security & Defense (5 modules)
10. Analysis & Reporting (7 modules)
11. Observability & Diagnostics (4 modules)
12. REST API Server (20 endpoints)
13. CLI Interface (28 commands)
14. TUI Dashboard (NexusApp)
15. Plugin System (9 hooks)
16. Integrations (6 platforms)
17. TUI Widgets (7 widgets)
18. Automation & Scheduling
19. Enterprise Features
20. Deployment Infrastructure (Docker / K8s / Compose)
21. Configuration & Constants
22. Data Models & Protocols
23. Algorithms & Detection Engines
24. Framework Mappings (MITRE / CWE / CAPEC / CVSS / OWASP)
25. Report Formats & Export (5 formats)
26. Testing Suite (31 files, 1,449 tests)
27. Documentation (14 documents, 5 ADRs)
28. Scripts & Tooling (4 scripts)
29. Engineering Reports (3 reports)
30. Complete Class Inventory
31. Complete Function Inventory
32. Final Statistics & Architecture Map

1. Executive Summary & Grand Totals

ReconPro v11.0 is an enterprise-grade reconnaissance and security analysis platform built entirely in pure Python with zero required external dependencies. The platform comprises 96 Python source files, 31 test files, and 29 deployment/documentation/script files, totaling approximately 156 files and 121,000+ lines of code. It features 26 scanning modules, 3 intelligence systems, a full REST API server with 20 endpoints, a Textual-based TUI dashboard, 28 CLI commands, 7 TUI widgets, 6 third-party integrations, 5 export formats, and 1,449 tests across 31 test files. The architecture follows a layered design: constants and utilities form the foundation, HTTP and connection layers provide network abstraction, the scan engine orchestrates module execution, and the CLI/TUI/API serve as presentation interfaces. A plugin system with 9 hooks enables extensibility, and the module registry pattern decouples 26 scanning modules from the core engine. The platform maps findings to MITRE ATT&CK, CWE, CAPEC, CVSS, DREAD, and OWASP frameworks. Deployment is supported via Docker, Docker Compose, and Kubernetes manifests with full enterprise security hardening.

Category	Count
Total Python Source Files	96
Total Test Files	31
Total Deployment/Docs/Script Files	29
Total Files in Project	156
Total Lines of Code	121,000+
Total Classes	85+
Total Standalone Functions	200+
Total CLI Commands	28
Total REST API Endpoints	20
Scanning Modules (Remote)	23
Scanning Modules (Local)	3
Total Modules	26
Plugin Hooks	9
Integrations	6
TUI Widgets	7
Report/Export Formats	5 (SARIF, JSON, Markdown, HTML, PDF)
Framework Mappings	6 (MITRE ATT&CK, CWE, CAPEC, CVSS, DREAD, OWASP)
Test Functions	1,449
Test Classes	291
Documentation Files	14 + 5 ADRs
Deployment Targets	3 (Docker, Docker Compose, Kubernetes)
Themed UI Themes	6 (Recon, Matrix, Cyber, Dark, Light, Ocean)
Configuration Constants	40+
Shared Utility Functions	17

2. Architecture Overview

ReconPro follows a layered architecture with strict separation of concerns. The foundation layer consists of constants.py (unified severity levels, score thresholds, color mappings, path defaults) and utils.py (17 shared pure functions for target normalization, scoring, grading, entropy calculation, and data sanitization). The network layer provides HTTP abstraction through http_layer.py (rate limiter, Finding dataclass, probe function), connection_pool.py (thread-safe urllib pool), async_http.py (aiohttp-based adaptive rate limiting with token-bucket EMA), and proxy.py (rotation strategies, Tor integration). The orchestration layer includes engine.py (async scan engine with bounded concurrency via asyncio.Semaphore), scanner.py (synchronous scan runner), and registry.py (centralized module registry with lazy imports). The module layer contains 26 scanning modules organized into remote (23 modules requiring network access) and local (3 modules for host auditing, developer security, and health checks). The presentation layer offers three interfaces: CLI via cli.py (argparse-based, 28 subcommands), TUI via nexus_tui.py (Textual framework, live dashboard), and REST API via server.py (stdlib HTTP server, 20 endpoints with auth). Cross-cutting concerns include observability (structured logging, metrics, tracing), security (input sanitization, audit logging), plugin system (9 lifecycle hooks), and memory management (unified store for findings, knowledge graph, credentials).

Layer	Components	Files
Foundation	Constants, Utilities, Interfaces, Context	constants.py, utils.py, interfaces.py, context.py
Network	HTTP Layer, Connection Pool, Async HTTP, Proxy, Raw Sockets	http_layer.py, connection_pool.py, async_http.py, proxy.py, raw_socket.py
Orchestration	Scan Engine, Scanner, Registry, Parallel	engine.py, scanner.py, registry.py, parallel.py
Modules (Remote)	23 scanning modules	modules/*.py (remote)
Modules (Local)	Host audit, Developer scan, Doctor	modules/host.py, dev.py, doctor.py
Intelligence	AI Analyst, Attack Graph, Threat Intel, Pipeline	ai_analyst.py, attack_graph.py, threat_intel.py, intelligence_pipeline.py
Analysis	Kill Chain, Attribution, Cognitive Sec, Ratings, Risk Choice	kill_chain.py, attribution.py, cognitive_sec.py, ratings.py, risk_choice.py
Reporting	Report Writer, Reports (HTML), Formats, Benchmark	report_writer.py, reports.py, formats.py, benchmark.py
Presentation	CLI, TUI, REST API, Chat, Wishes	cli.py, nexus_tui.py, server.py, chat.py, wishes.py
Observability	Structured Logger, Metrics, Tracer, Profiler, Telemetry	observability.py, profiler.py, telemetry.py
Security	Security Manager, Sanitize, MITM, Evasion, Defense	security.py, sanitize.py, mitm.py, evasion.py, defense.py
Plugins	Hook Manager, Plugin Discovery, Template	plugins.py
Integrations	Slack, Jira, GitHub, Splunk, PagerDuty, ZAI Stream	integrations/*.py
Memory	Finding Store, Agent Blackboard, Credential Vault, Knowledge	memory.py, knowledge_graph.py
Deployment	Dockerfile, Docker Compose, K8s manifests	deploy/*

3. Core Infrastructure

constants.py

Purpose: Single source of truth for severity levels, grade thresholds, color mappings, path defaults, and configuration constants used by all modules.

Functions: ['severity_sort_key(severity) -> int']

Constants: SEVERITY_LEVELS (5 levels), VALID_SEVERITIES (frozenset), GRADE_THRESHOLDS (7 grades: A+ through F), SEV_COLORS (5 mappings), GRADE_COLORS (7 mappings), SARIF_LEVEL_MAP, BADGE_COLOR_MAP, DEFAULT_TIMEOUT=8, DEFAULT_RATE_LIMIT=10.0, DEFAULT_BODY_LIMIT=16384, DEFAULT_MAX_WORKERS=4, USER_AGENT, DREAD_SCORE_MAP, MAX_SCORE=100, MIN_SCORE=0, RECONPRO_HOME, SCAN_HISTORY_DIR, PLUGIN_DIR, MEMORY_DIR, KNOWLEDGE_GRAPH_FILE, PLUGIN_HOOKS_FILE, TOOLS_DIR

Dependencies: stdlib (pathlib, __future__)

Status: Production | **Tests:** test_constants.py (44 tests), test_coverage_boost.py

utils.py

Purpose: Shared pure functions for target normalization, severity classification, scoring, grading, entropy calculation, and data utilities used across all modules.

Functions: extract_host, normalize_base_url, validate_target, count_severities, sort_findings_by_severity, severity_to_cvss, severity_to_dread, validate_severity, compute_score, compute_grade, badge_markdown, safe_int, safe_float, truncate, entropy, is_private_ip, url_join

Constants: None (pure functions)

Dependencies: internal (constants), stdlib (re, urllib.parse, typing, math)

Status: Production | **Tests:** test_utils.py (118 tests), test_property.py (58 tests), test_coverage_boost.py

registry.py

Purpose: Centralized module registry managing 23 remote and 3 local scanning modules with lazy imports, runner resolution, and metadata lookup.

Functions: build_module_registry, build_local_modules, get_module_runner, is_local_module, is_remote_module, get_module_info, list_remote_modules, list_local_modules, get_module_color

Constants: MODULE_REGISTRY (23 entries), LOCAL_MODULES (3: host, dev, doctor), ALL_MODULES (26), DEFAULT_MODULES (19), DEFAULT_LOCAL_MODULES (3)

Dependencies: internal (modules.* lazy), stdlib (typing)

Status: Production | **Tests:** test_registry.py (19 tests), test_integration.py

interfaces.py

Purpose: Protocol/ABC definitions establishing contracts for ScanModule, FindingProcessor, ReportGenerator, EventEmitter, ConfigurationProvider, and PluginInterface.

Functions: None (protocols only)

Constants: Type aliases: Findings, ScanResult, Severity, Grade, Target, ModuleID

Dependencies: stdlib (typing)

Status: Production | **Tests:** test_interfaces.py (79 tests)

context.py

Purpose: Unified scan context dataclass encapsulating all scan configuration parameters with serialization and timing support.

Functions: create_context(target, **kwargs) -> ScanContext
Constants: None
Dependencies: stdlib (dataclasses, time)
Status: Production | **Tests:** test_interfaces.py

memory.py

Purpose: Unified memory store with four subsystems: FindingStore (CRUD for findings), AgentBlackboard (key-value for agent state), CredentialVault (secure credential storage), and Knowledge Graph persistence.

Classes: FindingStore, AgentBlackboard, CredentialVault, UnifiedMemoryStore
Functions: UnifiedMemoryStore: add, query, recent, stats, save, load, clear
Constants: MEMORY_DIR, FINDINGS_DIR, VAULT_PATH
Dependencies: internal (knowledge_graph), stdlib (json, os, pathlib, hashlib)
Status: Production | **Tests:** test_enterprise.py

history.py

Purpose: Scan history management with save, list, diff, and cleanup operations for tracking scan results over time.

Functions: save_scan, list_scans, get_latest, get_scan, diff_scans, clear_history
Constants: HISTORY_DIR
Dependencies: stdlib (json, os, datetime, pathlib)
Status: Production | **Tests:** test_scanner.py

theme.py

Purpose: Unified theme system providing 6 color themes (Recon, Matrix, Cyber, Dark, Light, Ocean) for Rich console and Textual TUI rendering.

Functions: None (class-based properties)
Constants: THEMES (6 theme definitions with 20+ color values each)
Dependencies: internal (constants)
Status: Production | **Tests:** Embedded in TUI tests

4. Scanning Engine & Orchestration

The scanning engine is the core execution layer, responsible for orchestrating module runs against targets. It supports both synchronous (`scanner.py`) and asynchronous (`engine.py`) execution modes, as well as parallel multi-target blitz scanning (`parallel.py`). The engine uses bounded concurrency via `asyncio.Semaphore`, emits events for real-time monitoring, and supports plugin hooks at pre-scan, post-scan, and post-finding lifecycle points.

engine.py (726 lines)

Purpose: Async scan orchestration engine with bounded concurrency, event emission, and plugin hook integration.

Classes: `ScanEvent` (dataclass: `type`, `module_id`, `target`, `finding`, `findings_count`, `duration`, `result`, `modules`, `timestamp`), `EventCollector` (`findings`, `by_module`, `timeline`), `ScanEngine` (`_emit`, `_resolve_remote_modules`, `_resolve_local_modules`, `_build_result`, `_run_module`, `run`, `scan_one`)

Functions: `concurrent_scan(targets, **kw) -> Dict`

Key Algorithm: Bounded concurrency with `asyncio.Semaphore`, plugin hooks (`pre_scan`, `post_scan`, `post_finding`)

Tests: `test_scanner.py` (18 tests), `test_integration.py`

scanner.py (209 lines)

Purpose: Core synchronous scan orchestration running modules sequentially.

Classes: `ReconProResult` (dataclass: `target`, `findings`, `score`, `grade`, `module_results`, `elapsed`, `total_points_deducted`);
methods: `to_dict()`

Functions: `scan(target, **kwargs) -> ReconProResult`, `audit_scan(target, **kwargs) -> ReconProResult`

Tests: `test_scanner.py` (18 tests), `test_stress.py`

parallel.py (220 lines)

Purpose: Parallel multi-target scanning (blitz mode) using `ThreadPoolExecutor`.

Classes: None

Functions: `blitz_scan(targets, module_names, **kw)`, `_scan_one_target`, `_render_blitz_summary`

Key Algorithm: `ThreadPoolExecutor` + `done_callback` single-pass collection

Tests: `test_stress.py` (14 tests)

async_http.py (902 lines)

Purpose: Async HTTP engine with `aiohttp`-based adaptive rate limiting using token-bucket with EMA tracking.

Classes: `AsyncSession` (wraps `aiohttp.ClientSession` with cookie jars, redirect recording, sync fallback), `AdaptiveLimiter` (token-bucket with EMA tracking, hostname cache)

Functions: `async_probe`, `probe_sync`, `batch_probe`

Tests: `test_performance.py`

connection_pool.py (255 lines)

Purpose: Lightweight thread-safe HTTP connection pool using `urllib`.

Classes: `ConnectionPool` (`probe`, `stats`, `reset_stats`, `close`, `__repr__`)

Functions: None (class methods only)

Tests: `test_http_probe.py` (9 tests)

5. Network & Protocol Layer

http_layer.py (165 lines)

Purpose: Shared HTTP abstraction providing rate limiting, HTTP probing, and the universal Finding dataclass.

Classes: RateLimiter (`__init__`, `acquire`), Finding (dataclass: `title`, `severity`, `category`, `module`, `description`, `evidence`, `asset`, `points_deducted`, `remediation`, `dread_score`; methods: `to_dict`)

Functions: `http_probe(url, **kw)`, `compute_grade(score)`, `badge_markdown(grade)`

Constants: `UA`, `GRADE_MAP`, `default_limiter`

proxy.py (705 lines)

Purpose: Proxy pool management with 4 rotation strategies, health checking, Tor circuit integration, and proxy statistics.

Classes: Proxy (dataclass), RotationStrategy (Enum: `ROUND_ROBIN`, `RANDOM`, `LEAST_CONNECTIONS`, `GEO_DISTRIBUTED`), ProxyPool (`load_from_file`, `load_from_url`, `add_proxy`, `health_check`, `get_proxy`, `rotate`, `report_dead`, `report_success`, `stats`, `save`, `load`), TorProxyManager (`check_tor_available`, `new_circuit`, `get_tor_session`, `verify_new_ip`)

Functions: `_parse_proxy_str`, `_create_strategy`, `proxy_probe`, `load_default_pool`

raw_sockets.py (598 lines)

Purpose: Raw packet assembly for SYN scanning, UDP probing, TTL fingerprinting, and ARP discovery with optional scapy acceleration.

Classes: SynScanner (`scan`, `_scan_scapy`, `_scan_connect`), UdpProber (`probe`), TtlAnalyzer (`analyze`, `_estimate_initial_ttl`), ArpSpy (`discover`, `detect_spoofing`)

Functions: `_parse_ports`, `_grab_banner`, `_build_udp_probe`

Constants: `HAS_SCAPY`, `SERVICE_GUESSES (TCP)`, `UDP_SERVICE_GUESSES`, `TTL_SIGNATURES`

geoip.py (683 lines)

Purpose: IP geolocation via ip-api.com with local cloud provider CIDR matching for AWS, Azure, GCP, Cloudflare, and others.

Classes: GeoIPLookup (`enrich_ip`, `enrich_ips`, `save`, `clear_cache`, `get_cache_stats`), BatchGeoIP (`enrich_batch`, `get_stats`)

Functions: `is_hosting_ip`, `is_proxy_ip`, `format_geoip_summary`, `detect_cloud_provider`

Constants: `CLOUD_RANGES (AWS/Azure/GCP/Cloudflare/DO/Hetzner/Linode/OVH)`, `HOSTING_AS_PATTERNS`, `PROXY_AS_PATTERNS`

subdomains.py (441 lines)

Purpose: Subdomain enumeration from 11 OSINT sources including CT logs, securitytrails, censys, and DNS brute force with 100+ prefixes.

Classes: SubdomainResult (dataclass), SubdomainEnumerator (`enumerate`, `_source_crtsh`, `_source_certspotter`, `_source_censys`, `_source_bufferover`, `_source_riddler`, `_source_dns_bruteforce`, `_source_txt_enum`, `_source_wayback`, `_source_anubis`, `_source_securitytrails`, `_resolve_subdomains`)

Functions: `discover_ctlogs`, `discover_dns`, `discover_subdomains`

Constants: `DNS_BRUTE_PREFIXES (100+)`, `_ALL_SOURCES (11)`

tunnel_detect.py (1,025 lines)

Purpose: Protocol tunnel detection for 8 tunnel types: DNS, ICMP, IPv6, GRE, SSH, HTTP, HTTPS, TCP-over-UDP.

Classes: TunnelType (Enum: 8 types), TunnelIndicator (dataclass), EntropyAnalyzer (shannon_entropy, byte_entropy), DNSEntropyChecker, TunnelDetector (detect_all, check_dns_tunnel, check_icmp_tunnel, check_ipv6_tunnel, check_gre_tunnel, check_ssh_tunnel, check_http_tunnel, check_https_tunnel, check_tcp_over_udp)

6. Reconnaissance Modules – Standard

These modules comprise the standard reconnaissance toolkit covering surface-level web analysis, authentication testing, supply chain assessment, infrastructure profiling, and more. Each module follows the Finding dataclass contract and returns a list of Finding objects.

modules/recon.py (1,247 lines)

Purpose: 13-category surface reconnaissance including DNS, WHOIS, HTTP headers, SSL/TLS, ports, technology detection, robots.txt, sitemap, CORS, open redirects, and email harvesting.

Entry Point: `run_recon(target, base_url, timeout, verify_tls)`

modules/auth.py (1,038 lines)

Purpose: 15 authentication bypass techniques including SQL injection auth, default credentials, JWT manipulation, OAuth misconfig, session fixation, and cookie hijacking.

Entry Point: `run_auth(target, base_url, timeout, verify_tls)`

modules/chain.py (291 lines)

Purpose: SSRF and redirect chain hunting module wrapper for chain_engine.py.

Entry Point: `run_chain(target, base_url, timeout, verify_tls)`

modules/bot.py (355 lines)

Purpose: C2/bot infrastructure detection module.

Entry Point: `run_bot(target, base_url, timeout, verify_tls)`

modules/team.py (256 lines)

Purpose: Team collaboration module.

Entry Point: `run_team(target, base_url, timeout, verify_tls)`

modules/nhi.py (159 lines)

Purpose: Non-Human Identity blast radius mapping for service accounts, API keys, and machine identities.

Entry Point: `run_nhi(target, base_url, timeout, verify_tls)`

modules/host.py (1,488 lines)

Purpose: Full laptop/machine security audit (local module) covering OS hardening, network config, user accounts, installed software, cron jobs, and file permissions.

Entry Point: `run_host()`

modules/dev.py (1,087 lines)

Purpose: Developer security scan (local module) for secrets in code, dependency vulnerabilities, git history leaks, and Dockerfile issues.

Entry Point: `run_dev()`

modules/doctor.py (1,256 lines)

Purpose: Security health check with fix commands (local module) analyzing system configuration and providing remediation guidance.

Entry Point: `run_doctor()`

modules/ast_analyzer.py (611 lines)

Purpose: AST code analysis for Python, JavaScript, and TypeScript vulnerability patterns including injection, XSS, and insecure deserialization.

Entry Point: `run_ast_analyzer(target, base_url, timeout, verify_tls)`

Classes: `ASTAnalyzer` (`analyze_python`, `analyze_javascript`, `analyze_typescript`, `detect_patterns`, `generate_findings`)

modules/iac_audit.py (2,121 lines)

Purpose: Infrastructure-as-Code audit for Terraform, CloudFormation, Dockerfile, and Kubernetes manifests.

Entry Point: `run_iac_audit(target, base_url, timeout, verify_tls)`

Classes: `IaCAuditor` (`audit_terraform`, `audit_cloudformation`, `audit_dockerfile`, `audit_kubernetes`, `generate_report`)

modules/container_sec.py (1,018 lines)

Purpose: Container security analysis for Dockerfiles, Kubernetes manifests, and Docker Compose configurations.

Entry Point: `run_container_sec(target, base_url, timeout, verify_tls)`

Classes: `ContainerSecurityAnalyzer` (`analyze_dockerfile`, `analyze_k8s`, `analyze_compose`)

modules/cloud_recon.py (1,152 lines)

Purpose: Cloud infrastructure reconnaissance for AWS, Azure, and GCP metadata endpoints and asset discovery.

Entry Point: `run_cloud_recon(target, base_url, timeout, verify_tls)`

Classes: `CloudReconEngine` (`detect_aws`, `detect_azure`, `detect_gcp`, `scan_metadata`)

7. Advanced Reconnaissance Modules

The advanced modules represent ReconPro's cutting-edge capabilities, covering OS fingerprinting via HTTP timing, dark web monitoring, steganography detection, covert channel identification, zero-day hunting, infrastructure ghosting, signal intelligence, nation-state attribution, weaponized document analysis, honeypot detection, cryptographic dead drops, and more. These modules collectively provide intelligence-community-grade analysis capabilities.

Module	Lines	Funcs	Status	Key Capability
quantum_fingerprint.py	2026	14	Production	TCP/IP stack fingerprinting via 7 HTTP timing signals (TTL, TCP window, SYN-ACK)
dark_web_monitor.py	789	16	Production	Monitors 7 paste sites, 6 threat intel sources, 2 breach databases, and 13 cred
free_info_ops.py	2081	13	Production	Defensive information operations: infrastructure misattribution, false flag ris
steganography_detector.py	381	19	Production	10-category steganography detection: whitespace, base64 anomalies, header stego
covert_channel.py	1673	22	Production	8 covert channel types: DNS tunneling, HTTP header covert, timing, ICMP, certifi
zero_day_hunter.py	1576	14	Production	Anomaly-based zero-day detection: response anomalies, error message analysis, v
infrastructure_ghost.py	2116	22	Production	Complete digital infrastructure ghost: IP discovery, subdomain scanning (100+ p
signal_intelligence.py	2046	14	Production	Traffic analysis: beaconing detection, C2 pattern matching (Cobalt Strike, Meta
nation_state_attributor.py	769	10	Production	Nation-state attribution mapping to 20+ APT groups via TTP correlation, infrast
weaponized_report.py	2509	18	Production	Tracking element detection: tracking pixels (40+ signatures), beaconing URLs, s
honeypot_dance.py	1815	16	Production	Honeypot detection via 6 methods: response timing CV, error perfection, known f
dead_drop.py	2222	31	Production	8 cryptographic dead drop channels: DNS TXT, ETag, CT logs, HTTP headers, times
vibesec.py	206	1	Production	AI/Vibe-coding vulnerability benchmark: 7 categories, 100-point score, A+-F gra
pegasus.py	195	5	Production	Pegasus spyware detection: 116 C2 domains, 11 SMS lure patterns, 19 process sig
gorgon.py	634	19	Production	17-stage offensive scanner: SQLi/XSS/path traversal, HTTP methods, CORS, rate l
oblivion.py	749	27	Production	23-stage analytical dissolution: info disclosure, headers, JS secrets, mixed co

Total Advanced Module Lines: 23,887 lines | **Total Functions:** 271 | **Total Classes:** 3 (OSSignature, TimingSample, SubProbeResult) | **Signatures/Databases:** 21 OS signatures, 116 Pegasus C2 domains, 100+ stego signatures, 30+ fuzz payloads, 20+ APT groups, 100+ subdomain prefixes, 40+ tracker signatures

8. Intelligence Systems

ai_analyst.py (1,639 lines)

Purpose: AI Security Analyst with 6 sub-engines: FindingClassifier (24+ categories), FindingCorrelator (dedup, grouping, attack chain detection), ExploitabilityEstimator, BusinessImpactAnalyzer, AttackPathDetector, RemediationPrioritizer. Maps to MITRE ATT&CK; (30+ techniques), CWE (25+), CAPEC (20+).

Classes: FindingClassifier, FindingCorrelator, ExploitabilityEstimator, BusinessImpactAnalyzer, AttackPathDetector, RemediationPrioritizer, AIAnalystEngine, ExtendedFinding (dataclass)

Functions: run_ai_analyst, classify_finding, correlate_findings, suggest_attack_paths, generate_report, analyze

Tests: test_ai_analyst.py (55 tests)

attack_graph.py (988 lines)

Purpose: Attack graph engine building directed graphs from scan findings to identify attack paths, lateral movement, privilege escalation, choke points, and blast radius.

Classes: GraphNode (dataclass), GraphEdge (dataclass), AttackGraph (add_node, add_edge, build_from_findings, find_attack_paths, find_lateral_movement, find_privilege_escalation, find_choke_points, blast_radius, riskiest_nodes, save, load, to_dict, stats)

Functions: run_attack_graph (implied)

Tests: test_attack_graph.py (24 tests)

threat_intel.py (940 lines)

Purpose: Threat Intelligence Center providing CVE, CWE, CAPEC, and MITRE ATT&CK; enrichment for scan findings.

Classes: ThreatIntelCenter (lookup_cve, lookup_cwe, lookup_capec, lookup_mitre, enrich_finding)

Functions: run_threat_intel

Tests: test_threat_intel.py (48 tests)

threat_feeds.py (938 lines)

Purpose: Threat feed aggregation from multiple OSINT sources with parsing, deduplication, and alerting.

Classes: ThreatFeedAggregator (fetch_feeds, parse_feed, aggregate, get_threats)

Functions: run_threat_feeds

intelligence_pipeline.py (456 lines)

Purpose: Post-scan intelligence pipeline composing AI analyst, attack graph, and threat intel into unified intelligence results.

Classes: IntelligenceResult (dataclass), IntelligencePipeline (analyze)

Functions: run_intelligence_pipeline (implied)

Tests: test_intelligence_pipeline.py (32 tests)

knowledge_graph.py (774 lines)

Purpose: Directed security knowledge graph with 10 node types, 15+ edge types, shortest path, blast radius, attack chains, connected components, and NetworkX acceleration.

Classes: SecurityKnowledgeGraph (add_node, add_edge, remove_node, get_node, get_neighbors, shortest_path, blast_radius, attack_chains, connected_components, save, load, stats, to_dict)

Functions: N-A

social_graph.py (935 lines)

Purpose: OSINT social graph intelligence extracting entities and relationships from DNS, WHOIS, HTTP, and CT log data with community detection.

Classes: Entity (dataclass), Relationship (dataclass), RelationshipExtractor, SocialGraph, GraphIntelligence

Functions: N-A

9. Security & Defense

security.py (811 lines)

Purpose: Security utilities providing input sanitization, auth token management, secret detection, audit logging, file hash computation, and threat classification.

Classes: SecurityManager (sanitize_input, validate_token, generate_token, check_permissions), SecurityAuditLogger

Functions: run_security_check, sanitize_target, sanitize_path, sanitize_filename, sanitize_html, sanitize_shell, sanitize_log, detect_secrets_in_text, safe_json_parse, safe_url_parse, safe_xml_parse, compute_file_hash, verify_module_signature, check_dependency_integrity, get_security_profile, classify_threat

Tests: test_security.py (90 tests), test_security_regression.py (43 tests), test_security_hardening.py (30 tests)

sanitize.py (32 lines)

Purpose: Minimal input sanitization for HTML entities and path/command injection prevention.

Classes: N-A

Functions: sanitize_input, sanitize_path, sanitize_command

mitm.py (612 lines)

Purpose: MITM analysis checking certificate pinning, HSTS enforcement, and TLS configuration.

Classes: MITMAnalyzer (check_cert_pinning, check_hsts, check_tls_config, analyze)

Functions: run_mitm

evasion.py (1,117 lines)

Purpose: Evasion techniques for WAF bypass, payload encoding, request fragmentation, and variant generation.

Classes: EvasionEngine (encode_payload, fragment_request, bypass_waf, generate_variants)

Functions: run_evasion

defense.py (1,402 lines)

Purpose: Generates remediation code including Flask application builders with security fixes, WAF rules, security headers, and CSP policies.

Classes: DefenseGenerator (generate_code, _generate_flask, _generate_waf, _generate_headers, _generate_csp)

Functions: run_defense

10. Analysis & Reporting

kill_chain.py (2,992 lines)

Purpose: Full MITRE ATT&CK; kill chain analysis mapping findings to attack phases.

Classes: KillChainAnalyzer

Functions: run_kill_chain

attribution.py (3,144 lines)

Purpose: Nation-state attack attribution engine with TTP-based attribution matching.

Classes: AttributionEngine

Functions: run_attribution

cognitive_sec.py (1,049 lines)

Purpose: Cognitive security analysis detecting bias, manipulation patterns, and influence operations.

Classes: CognitiveSecurityAnalyzer

Functions: run_cognitive_sec

ratings.py (574 lines)

Purpose: DREAD scoring engine and risk rating computation.

Classes: RatingsEngine (rate_finding, rate_target, compute_dread, get_risk_level)

Functions: run_ratings

compliance.py (1,013 lines)

Purpose: Compliance framework mapping to ISO 27001, NIST 800-53, SOC2, PCI-DSS, CIS, and GDPR.

Classes: ComplianceMapper (map_findings, generate_report, _map_iso27001, _map_nist, _map_pci, _map_cis, _map_gdpr)

Functions: run_compliance

cross_validator.py (527 lines)

Purpose: Cross-validation of findings: deduplication, correlation, and confidence scoring.

Classes: CrossValidator (validate, deduplicate, correlate, confidence_score)

Functions: run_cross_validator

benchmark.py (485 lines)

Purpose: Score tracking and benchmarking with historical trends.

Classes: BenchmarkTracker (record, get_history, get_trend, compare, generate_report)

Functions: run_benchmark

11. Observability & Diagnostics

observability.py (1,002 lines)

Purpose: Enterprise observability: structured logging (JSON), metrics collection (counter/gauge/histogram/timer), health checks, performance counters, scan tracing.

Classes: StructuredLogger, MetricsCollector, ScanTracer, Profiler, HealthMonitor, TelemetryManager

Functions: Class methods only

Tests: test_observability.py (132 tests)

profiler.py (791 lines)

Purpose: Performance profiling with module timing, memory tracking, HTTP request counting, and report generation.

Classes: ScanProfiler (start, stop, record, get_profile, generate_report)

Functions: run_profiler

diagnostics.py (484 lines)

Purpose: System diagnostics: Python version, OS, network, permissions, module status (26+ modules), config validation, debug report generation.

Classes: DiagnosticRunner (run_all, check_python, check_os, check_network, check_permissions, generate_report)

Functions: run_diagnostics

Tests: test_diagnostics.py (49 tests)

telemetry.py (207 lines)

Purpose: Anonymous telemetry collection with opt-out support and install ID tracking.

Classes: N-A

Functions: telemetry_enabled, send_telemetry, disable_telemetry

12. REST API Server (20 Endpoints)

The REST API server is implemented in `server.py` using Python's `stdlib` `BaseHTTPRequestHandler`. It provides 20 endpoints covering scan execution, history management, module listing, agent control, report generation, subdomain enumeration, authentication, CVE lookup, and export functionality. Authentication uses token-based auth with bootstrap secret for initial setup. SSE (Server-Sent Events) are supported for real-time streaming via the `/events` endpoint.

Method	Endpoint	Purpose	Auth
GET	/	API status and health check	No
POST	/scan	Execute standard scan	Yes
POST	/audit	Execute audit scan	Yes
POST	/blitz	Execute parallel multi-target scan	Yes
GET	/history	List scan history	Yes
GET	/history/<filename>	Get specific scan result	Yes
GET	/modules	List available modules	No
POST	/agent	Run Nexus AI agent	Yes
POST	/report	Generate report	Yes
GET	/report/<filename>	Get generated report	Yes
GET	/subdomains/<domain>	Enumerate subdomains	Yes
GET	/auth/status	Authentication status	No
POST	/auth/token	Generate API token	Bootstrap
POST	/auth/revoke	Revoke API token	Yes
GET	/auth/tokens	List active tokens	Yes
GET	/passive/<domain>	Passive intelligence	Yes
GET	/cve/<cve_id>	CVE lookup	Yes
POST	/scan/vibesec	Execute vibesec scan	Yes
POST	/export/csv	Export findings as CSV	Yes
POST	/export/sarif	Export findings as SARIF	Yes
GET	/events	Server-Sent Events stream	Yes

Server Functions: `generate_api_token`, `validate_api_token`, `revoke_api_token`, `list_api_tokens`, `start_server`, `run_server`

Server Classes: `_Handler(BaseHTTPRequestHandler)` with `do_GET`, `do_POST`, `do_OPTIONS`, `_json_response`, `_read_body`, `_check_auth`, `_unauthorized`, `_forbidden`, `log_message`

13. CLI Interface (28 Commands)

The CLI interface is implemented in `cli.py` (2,013 lines) using `argparse` with Rich console rendering. It provides 28 subcommands, ASCII banner art, severity-colored output, findings table rendering, module breakdown visualization, badge display, remediation listing, and JSON output support. Help text is enhanced via `cli_help.py` (1,003 lines) with per-module examples, descriptions, and cross-references.

#	Command	Purpose
1	scan	Execute standard reconnaissance scan
2	vibsec	Execute AI/Vibe-coding vulnerability benchmark
3	audit	Execute audit scan
4	dev	Execute developer security scan
5	doctor	Run security health check
6	ports	Port scanning
7	secrets	Secret detection scanning
8	list	List available modules
9	nexus	Launch Nexus TUI dashboard
10	chat	Interactive REPL chat
11	tui	Textual TUI interface
12	blitz	Parallel multi-target scan
13	agent	Run Nexus AI agent
14	subdomains	Subdomain enumeration
15	schedule	Manage scan schedules
16	serve	Start REST API server
17	report	Generate reports
18	history	View scan history
19	diff	Diff scan results
20	screenshot	Take website screenshots
21	open	Open reports in browser
22	plugin	Plugin management
23	swarm	Multi-agent swarm attack
24	adversarial	Adversarial self-play
25	ast	AST code analysis
26	cve	CVE radar lookup
27	graph	Attack graph visualization
28	iac	Infrastructure-as-Code audit

14. TUI Dashboard (NexusApp)

The Textual-based TUI dashboard (nexus_tui.py, 3,359 lines) provides a rich terminal interface with live findings display, module grid, integrated chat, agent control, and real-time statistics. It uses the NexusApp class (extending Textual App) with reactive widgets, tab navigation, and keyboard shortcuts. The HelpOverlay (nexus_help.py, 203 lines) provides visual help with 14 keybindings and 5 command groups.

NexusApp Methods: compose, on_mount, on_input_submitted, _start_agent_scan, _update_modules, _handle_finding, _run_scan, _run_blitz, _run_agent, _run_passive, _run_fuzzer, _run_cve, _run_export, _run_defense, _run_compliance, _run_audit, _run_subdomains, _run_history, _run_diff, _run_benchmark, _run_graph, _toggle_chat, _on_tab, on_key (+ many more reactive methods)

15. Plugin System (9 Hooks)

The plugin system (plugins.py, 300 lines) provides lifecycle hook management and dynamic plugin loading from ~/.reconpro/plugins/. Plugins can register for 9 lifecycle hooks and are discovered automatically at startup.

Hook	Trigger Point	Use Case
pre_scan	Before scan execution	Target validation, rate limiting setup
post_scan	After scan completion	Report generation, alerting
pre_finding	Before finding is recorded	Finding enrichment, filtering
post_finding	After finding is recorded	Real-time alerts, webhooks
pre_report	Before report generation	Custom report templates
post_report	After report generation	Report distribution
new_target	When new target is added	Target preprocessing
vulnerability	When vulnerability is found	Immediate remediation triggers
error	When error occurs	Error handling, logging

Classes: HookManager (register, unregister, fire, clear, list_hooks, save_hooks, load_hooks)
Functions: _ensure_plugin_dir, discover_plugins, run_plugin, create_plugin_template, register_plugin, get_registered_plugins, load_all_plugins

16. Integrations (6 Platforms)

Platform	File	Lines	Capabilities	Status
Slack	integrations/slack.py	411	send_message, send_findings, send_alert, configure	Production
Jira	integrations/jira.py	391	create_ticket, update_ticket, search_tickets, configure	Production
GitHub	integrations/github.py	508	create_issue, create_pr, search_code, configure	Production
Splunk	integrations/splunk.py	54	send_event, configure	Stub
PagerDuty	integrations/pagerduty.py	72	create_incident, configure	Stub
ZAI Stream	integrations/zai_stream.py	428	stream_event, stream_findings, configure	Production

17. TUI Widgets (7 Widgets)

Widget	File	Lines	Purpose
ScoreGauge	widgets/score_gauge.py	238	Animated score gauge with grade display
Toast	widgets/toast.py	211	Toast notification popup with auto-dismiss
HintBar	widgets/hint_bar.py	345	Contextual hint bar with action text
StatCounter	widgets/stat_counter.py	205	Animated statistic counter
VelocityMeter	widgets/velocity_meter.py	235	Findings velocity meter (findings/min)
Sparkline	widgets/sparkline.py	281	Inline sparkline chart widget
CommandCompleter	widgets/command_completer.py	558	Auto-completer with suggestion popup

18. Automation & Scheduling

scheduler.py (105 lines): Cron-like recurring scan scheduler with `schedule_scan`, `list_schedules`, `cancel_schedule` functions. Scans run in background threads at specified intervals.

wishes.py (1,423 lines): Natural language wish fulfillment engine parsing commands like 'scan example.com for XSS' into structured scan parameters. `WishEngine` class with `parse`, `fulfill`, `_wish_scan`, `_wish_audit`, `_wish_export`, `_wish_report`, `_wish_history`, `_wish_diff` methods.

chat.py (447 lines): Interactive REPL with `ChatSession` class processing natural language commands. Handles `scan`, `audit`, `blitz`, `help`, `export`, `modules`, and `history` commands.

swarm.py (1,120 lines): Multi-agent attack swarm with 4 phases: SCOUT (recon), HACKER (attack), CODER (remediation), GUARDIAN (defense). `SwarmAgent` class with role-based execution.

adversarial.py (1,150 lines): Adversarial self-play engine with 3 phases: hacker (attack), coder (fix), verify (validate). `AdversarialEngine` class with `run`, `_hacker_phase`, `_coder_phase`, `_verify_phase` methods.

webhooks.py (875 lines): Webhook management with `register`, `unregister`, `fire`, `process`, `list_webhooks`. `WebhookManager` class for both incoming and outgoing webhooks.

collab.py (531 lines): Collaboration manager for shared scans with `share_scan`, `list_shared`, `get_shared`, `delete_shared`. `CollabManager` class.

19. Enterprise Features

Memory Management (memory.py, 902 lines): Unified memory store with `FindingStore` (CRUD), `AgentBlackboard` (key-value), `CredentialVault` (secure storage), and `KnowledgeGraph` persistence. All subsystems support `save`, `load`, `clear`, and `stats` operations.

Knowledge Graph (knowledge_graph.py, 774 lines): Directed graph with 10 node types and 15+ edge types. Supports `shortest_path`, `blast_radius`, `attack_chains`, `connected_components`. `NetworkX` acceleration when available.

Social Graph (social_graph.py, 935 lines): OSINT entity/relationship extraction from DNS, WHOIS, HTTP, CT logs. Community detection and cluster analysis.

API Discovery (api_discovery.py, 1,184 lines): API Blueprint reconstruction with 5 engines: `JsEndpointExtractor`, `GraphQLIntrospector`, `OpenAPISpecParser`, `LinkCrawler`, `AuthMatrix`, `GrpcIntrospector`.

Exfil Channels (exfil_channels.py, 973 lines): Data exfiltration channel mapper identifying 9 channel types with viability scoring, bandwidth estimation, stealth scoring, and risk assessment.

Shadow IT (shadow_it.py, 1,684 lines): Shadow IT and abandoned infrastructure discovery with decay indicators, default page detection, certificate age analysis, and asset classification.

Supply Chain (supply_chain.py, 973 lines): Supply chain analysis with `WebExtractor`, `GitHubScraper`, and `SupplyChainAnalyzer` for secrets, dependencies, infrastructure, CI/CD, and cloud config.

Drift Monitor (drift_monitor.py, 939 lines): Infrastructure drift detection across 8 categories: DNS, TLS certs, HTTP headers, tech stack, ports, security headers, content, status codes.

Netmap (netmap.py, 723 lines): Zero-trust network mapper with host discovery, trust mapping, segmentation checking. `NetworkDiscovery`, `TrustMapper`, `SegmentationChecker` classes.

Network Graphs: GraphRenderer (graph_ui.py, 677 lines) for text-based graph rendering: tree, digraph, table, ASCII graph formats.

Tool SDK (tool_sdk.py, 728 lines): External tool integration framework for nmap, nuclei, and other tools. ToolManager with register_tool, run_tool, parse_output, list_tools.

Browser Module (browser_mod.py, 158 lines): Playwright-based screenshot capture.

ANSI Capture (ansi_capture.py, 216 lines): ANSI escape code capture/strip for TUI.

Fuzzer (fuzzer.py, 1,743 lines): Parameter fuzzer for URL/path/header fuzzing with Fuzzer class (fuzz_url, fuzz_params, fuzz_headers, fuzz_paths, run).

Passive Intel (passive_intel.py, 540 lines): Passive intelligence collection without active scanning: DNS, WHOIS, HTTP headers. PassiveIntelCollector class.

CVE Radar (cve_radar.py, 393 lines): CVE radar for finding enrichment with NVD data.

AI CVE DB (ai_cve_db.py, 495 lines): Local CVE/NVD knowledge base with AICVEDB class.

AI Red Team (ai_red_team.py, 932 lines): AI red team module with GORGON 15-stage engine.

Entropy (entropy.py, 727 lines): Granular secret entropy scoring with Shannon entropy and contextual signals. SecretClassifier, ContextAnalyzer classes.

20. Deployment Infrastructure

Component	File	Key Configuration
Docker	deploy/Dockerfile	Base: python:3.12-slim, Multi-stage, Non-root user (1001), Port 7890, Healthcheck, Labels
Docker Compose	deploy/docker-compose.yml	Image: reconpro:v11-enterprise, Port 7890, Auth required, Volume: reconpro-data, CPU
K8s Deployment	deploy/k8s/deployment.yaml	Namespace: reconpro, Replicas: 3, Rolling update, Security context (non-root, seccomp
K8s Service	deploy/k8s/service.yaml	ClusterIP, Port 7890/TCP, Topology hints: auto
K8s ConfigMap	deploy/k8s/configmap.yaml	RECONPRO_REQUIRE_AUTH=true, LOG_LEVEL=INFO, MAX_WORKERS=4, DEFAULT_TIMEOUT=8
K8s Secret	deploy/k8s/secret.yaml.example	RECONPRO_BOOTSTRAP_SECRET (generated via openssl rand -hex 32)

Docker ENV vars: PYTHONUNBUFFERED=1, PYTHONDONTWRITEBYTECODE=1, PYTHONPATH=/opt/reconpro, RECONPRO_DATA_DIR=/home/reconpro/.reconpro, RECONPRO_LOG_LEVEL=INFO, RECONPRO_MAX_WORKERS=4, RECONPRO_DEFAULT_TIMEOUT=8

Docker Compose security: no-new-privileges:true, cap_drop: ALL, cap_add: NET_BIND_SERVICE, Network: reconpro-enterprise-net (bridge, 172.28.0.0/16)

K8s security: runAsNonRoot=true, runAsUser=1001, fsGroup=1001, seccompProfile=RuntimeDefault, automountServiceAccountToken=false, drop ALL capabilities, no privilege escalation

21. Configuration & Constants

Constant	Value	Purpose
DEFAULT_TIMEOUT	8 seconds	HTTP request timeout
DEFAULT_RATE_LIMIT	10.0 requests/sec	Rate limiter default
DEFAULT_BODY_LIMIT	16384 bytes (16KB)	HTTP response body limit
DEFAULT_MAX_WORKERS	4	Default parallel workers
MAX_SCORE	100	Maximum security score
MIN_SCORE	0	Minimum security score
USER_AGENT	ReconPro/11.0.0	Default HTTP user agent
SEVERITY_LEVELS	5 (Critical, High, Medium, Low, Info)	Severity classification
GRADE_THRESHOLDS	7 (A+ through F)	Score-to-grade mapping
SEV_COLORS	5 mappings	Severity color codes
GRADE_COLORS	7 mappings	Grade color codes
SARIF_LEVEL_MAP	5 mappings	SARIF severity levels
BADGE_COLOR_MAP	7 mappings	Badge color codes
DREAD_SCORE_MAP	6 mappings	DREAD severity-to-score
RECONPRO_HOME	~/.reconpro	Application home directory
SCAN_HISTORY_DIR	~/.reconpro/history	Scan history storage
PLUGIN_DIR	~/.reconpro/plugins	Plugin directory
MEMORY_DIR	~/.reconpro/memory	Memory store directory
KNOWLEDGE_GRAPH_FILE	~/.reconpro/memory/kg.json	Knowledge graph persistence

22. Data Models & Protocols

Model	File	Fields	Purpose
Finding	http_layer.py	title, severity, category, module, description, remediation, status, deducted, remediation	Universal finding data class
ScanEvent	engine.py	type, module_id, target, finding, findings, scan_id, scan_type, result, modules, timestamp	Scan lifecycle events
ReconProResult	scanner.py	target, findings, score, grade, module_results, status, remediation, points_deducted	Scan results and statistics
ScanContext	context.py	target, modules, timeout, verify_tls, rate_limit, configurations, +computed fields	Scan configuration and context
OSSignature	quantum_fingerprint.py	name, family, ttl_initial, tcp_window, syn_ack_timeout, keepalive_s, congestion, mss	OS fingerprinting signatures
TimingSample	quantum_fingerprint.py	label, elapsed_ms, payload_size, status_code, response_size, error	Timing probe samples
SubProbeResult	quantum_fingerprint.py	probe_name, samples, derived_value, description, properties: values, mean, median	Probe aggregation results
SecretMatch	entropy.py	Match details for secret detection	Secret detection match
SubdomainResult	subdomains.py	domain, subdomains, resolved, sources_used, status, unique_ips, resolved	Subdomain enumeration results
DriftSnapshot	drift_monitor.py	timestamp, target, dns, cert, headers, tech_stack, structure, content, status	Infrastructure drift snapshot
DriftEvent	drift_monitor.py	category, description, old_value, new_value, drift_score, event	Drift change event

Model	File	Fields	Purpose
ExfilChannel	exfil_channels.py	channel_type, viability_score, bandwidth, etc	Exfiltration channels
Entity	social_graph.py	entity_id, entity_type, name, properties	Social graph entity
Relationship	social_graph.py	source, target, rel_type, weight, properties	Social graph relationship
GraphNode	attack_graph.py	id, node_type, label, severity, risk_score, etc	Attack graph node
GraphEdge	attack_graph.py	source, target, edge_type, weight	Attack graph edge
IntelligenceResult	intelligence_pipeline.py	Composite intelligence analysis result	Pipeline output
ExtendedFinding	ai_analyst.py	Extended finding with MITRE/CWE/CAPEC/CVSS/Enriched finding	Extended finding
Proxy	proxy.py	Proxy connection details	Proxy dataclass
TunnelIndicator	tunnel_detect.py	Tunnel detection indicators	Tunnel detection result

23. Algorithms & Detection Engines

Algorithm	Module	Description
HTTP Timing OS Fingerprinting	quantum_fingerprint	7-signal OS identification: TTL deduction (CV-slope), TCP window (transfer-rate)
Shannon Entropy Analysis	entropy, covert_channel, isfingerprint	Information theoretic entropy calculation for detecting encrypted/encoded data
Token-Bucket Rate Limiting	async_http	Adaptive rate limiting with exponential moving average tracking and per-hostnam
Beaconing Detection	signal_intelligence	Jitter analysis, interval regularity scoring, autocorrelation for periodicity,
C2 Pattern Matching	signal_intelligence, gorilla	Response header/cookie/URI pattern matching against Cobalt Strike, Metasploit,
TTP Correlation	nation_state_attributor, Jaccard	Similarity between observed techniques and known APT group TTP database
Attack Path Discovery	attack_graph	Directed graph traversal finding chains from initial access to data exfiltratio
Blast Radius Computation	attack_graph, knowledge_base	Graph-based blast radius from any node to identify potentially compromised assets
Coefficient of Variation	honeypot_dance, signal_intelligence	Statistical measure (std/mean) for detecting abnormally consistent responses (h
Z-Score Anomaly Detection	zero_day_hunter	Statistical baseline comparison for response time, size, and status code anomal
Chi-Squared Frequency Analysis	steganography_detector	Statistical test for detecting non-uniform character distributions indicating h
Index of Coincidence	covert_channel	Cryptanalysis technique for detecting language/frequency anomalies in encoded c
Version-Response Correlation	zero_day_hunter	Version string extraction with known CVE database cross-referencing
Infrastructure Drift Hashing	infrastructure_ghost, drift_detector	Hash-based comparison of infrastructure snapshots for detecting unauthorized ch
DNS TXT Dead Drop Detection	dead_drop	High-entropy TXT record analysis with encoding detection (base64/hex/base32)
ETag Covert Channel Analysis	dead_drop	ETag header entropy and encoding pattern detection for covert data channels
Error Perfection Analysis	honeypot_dance	Comparison of error responses against known-real-server patterns to detect "too
DREAD Composite Scoring	ratings, zero_day_hunter	Damage, Availability, Reproducibility, Exploitability, Affected users, Discoverability weight
Community Detection	social_graph	Graph community detection algorithms for identifying related infrastructure clu
Proxy Rotation Strategies	proxy	4 strategies: Round Robin, Random, Least Connections, Geo-Distributed with heal
Parallel Scan Orchestration	engine, parallel	ThreadPoolExecutor + done_callback (parallel), asyncio.Semaphore (async engine)

24. Framework Mappings

ReconPro maps scan findings to 6 industry-standard security frameworks, enabling compliance reporting and integration with enterprise security tools:

Framework	Module	Coverage	Purpose
MITRE ATT&CK	ai_analyst, kill_chain, compliance, threat_intel	30+ tactics, techniques mapped	Tactics, techniques, and procedures classification
CWE	ai_analyst, threat_intel	25+ weaknesses mapped	Common Weakness Enumeration
CAPEC	ai_analyst, threat_intel	20+ patterns mapped	Common Attack Pattern Enumeration
CVSS	ai_analyst, formats, utils	Full scoring support	Common Vulnerability Scoring System
DREAD	ratings, zero_day_hunter, obfuscation, severity_levels	Damage, Reproducibility, Exploitability, Affected, Discoverability	
OWASP	compliance, formats	Top 10 covered	Web application security risks
ISO 27001	compliance	Control mapping	Information security management
NIST 800-53	compliance	Control mapping	Security and privacy controls
SOC 2	compliance	Trust criteria mapping	Service organization controls
PCI-DSS	compliance	Requirement mapping	Payment card data security
CIS	compliance	Benchmark mapping	Center for Internet Security
GDPR	compliance	Article mapping	General Data Protection Regulation
SARIF	formats	Full export	Static Analysis Results Interchange Format

25. Report Formats & Export

ReconPro supports 5 export formats for scan results, each optimized for different use cases:

Format	Module	Engine	Purpose
SARIF	formats.py	stdlib json	Static Analysis Results Interchange Format for GitHub/GitLab integration
JSON	formats.py	stdlib json	Structured data export for programmatic consumption
Markdown	formats.py	stdlib string	Human-readable report with severity badges and tables
HTML	reports.py	Chart.js + CSS	Self-contained interactive report with 8 Chart.js charts, glassmorphism CSS
PDF	formats.py	Print-ready HTML	Print-optimized HTML formatted for PDF conversion
AI Markdown	report_writer.py	openai (optional)	AI-generated audience-specific reports (Executive, Technical, Compliance, Developer)
HTML Dashboard	reports.py	Chart.js	Animated counters, severity distribution, category breakdown, top findings, timeline

HTML Report Features: 8 Chart.js charts (severity distribution pie, category bar, module horizontal bar, timeline line, top findings, score gauge, DREAD radar, severity heatmap), glassmorphism CSS theme, animated counters, responsive layout, dark mode support, self-contained (no external dependencies except Chart.js CDN).

AI Report Writer: 4 audience modes (Executive, Technical, Compliance, Developer) with LLM-generated narrative or template-based fallback. Severity sorting, emoji badges, remediation prioritization.

26. Testing Suite (31 Files, 1,449 Tests)

The test suite spans 31 files with approximately 1,449 individual test functions organized into 291 test classes. Tests cover unit, integration, stress, performance, security, regression, property-based, and coverage boost categories. The test runner (run_tests.py) provides automated discovery and execution.

Test File	Lines	Tests	Type	Coverage Area
test_utils.py	578	118	Unit	All 16 utility functions
test_observability.py	1179	132	Unit	Structured Logger, Metrics, Tracer, Profiler, Health, Telemetry
test_coverage_boost.py	737	110	Coverage	Edge cases for all utilities, constants, Finding
test_ai_analyst.py	729	55	Unit/Integration	FindingClassifier, Correlator, AttackPath, Mappings
test_security.py	992	90	Security	15 security functions: sanitize, detect, audit, classify
test_interfaces.py	682	79	Unit	All Protocol classes, type aliases, ScanContext
test_formats.py	744	50	Unit/Integration	SARIF, Markdown, JSON, PDF exports
test_diagnostics.py	612	49	Unit	Diagnostics, health check, version info, module status
test_constants.py	220	44	Unit	All constant values and mappings
test_threat_intel.py	306	48	Unit/Integration	ThreatIntel databases, enrichment pipeline
test_security_regression.py	761	43	Security/Regression	10 injection categories against all entry points
test_cli.py	499	32	Unit	CLI rendering functions, colors, badges, summaries
test_intelligence_pipeline.py	367	32	Integration	Full intelligence pipeline end-to-end
test_property.py	475	58	Property-based	Mathematical properties of core functions
test_enterprise.py	564	34	Integration	Intelligence engines, export formats, observability
test_attack_graph.py	306	24	Unit	DiGraph, classification, kill chain, blast radius
test_security_hardening.py	417	30	Security/Hardening	Path traversal, XSS, auth, error sanitization
test_performance.py	799	16	Performance	Startup, HTTP probe, scan, rate limiter benchmarks
test_scanner.py	177	18	Integration	scan(), audit_scan(), ReconProResult
test_stress.py	376	14	Stress	1000-10000 finding lists, long targets, memory
test_input_validation.py	180	18	Security	SQLi, command injection, path traversal, XSS, null bytes
test_regression_v11.py	409	36	Regression	Score bounds, severity ordering, grade thresholds
test_registry.py	169	19	Unit	Module registry counts, runners, structure
test_plugins.py	181	16	Unit	HookManager register/fire/clear/unregister
test_rate_limiter.py	112	7	Unit/Concurrency	Ratelimiter timing and thread safety
test_http_probe.py	200	9	Unit (mocked)	http_probe response structure, errors, limiter
test_finding.py	119	9	Unit	Finding creation, defaults, to_dict
test_integration.py	177	10	Integration	All 26 modules importable, runners callable
test_scoring.py	127	14	Unit	Score calculation, clamping, grade integration

27. Documentation (14 Documents, 5 ADRs)

Document	Lines	Key Sections
ARCHITECTURE.md	240	Design Principles, Package Structure, Core Components, Data Flow, Error Handling, Extensibility
DEPLOYMENT_GUIDE.md	148	Installation (pip/wheel/source), Configuration, Enterprise (Air-Gap, Container, CI/CD)
DEVELOPER_GUIDE.md	308	Prerequisites, First Scan, Custom Module Tutorial, Plugin Development, Testing, Contributing
MODULE_GUIDE.md	294	Core Remote Modules (10), Advanced Modules (12), Local Modules (3), Reference Table
API_REFERENCE.md	351	Public API, HTTP Layer, Utilities, Security, Observability, Formats
SECURITY_MODEL.md	204	Threat Model, Security Architecture, Hardening Guide, Known Limitations
PERFORMANCE_GUIDE.md	159	Benchmarks, Startup/Scan/Memory/CPU, Optimization Tips, Scaling
RESEARCH.md	483	11-tool Comparison Matrix, MITRE ATT&CK Coverage, Unique Capabilities, Architecture Patterns
ROADMAP.md	127	v10.0.0 (Current), v10.1.0 (Next), v11.0.0 (Future), Contributing
ADR-001	80	Pure Python Architecture decision
ADR-002	98	Centralized Module Registry decision
ADR-003	116	HTTP Timing Fingerprinting decision
ADR-004	104	Universal Finding Dataclass decision
ADR-005	120	Zero-Config Operation decision

28. Scripts & Tooling

Script	Lines	Purpose
generate_engineering_report.py	860	Generate v10 Enterprise Engineering Report as PDF via ReportLab
benchmark_intelligence.py	304	Benchmark intelligence pipeline (AI analyst, attack graph, threat intel)
benchmark_v11.py	297	v11 performance benchmarking at multiple finding scales
generate_v11_report.py	590	Generate v11 production readiness report

29. Engineering Reports

Report	Format	Key Findings
team1_architecture_review.md	Markdown (654 lines)	25 issues (2 critical, 6 high, 7 medium, 10 low). Critical: dual scan engines, UA inc
team2_benchmark_data.json	JSON	15 benchmarks: JSON/SARIF/Markdown/PDF export at 100/1000/10000 findings; intelligenc
team3_security_audit.md	Markdown (490 lines)	25 findings (3 critical, 6 high, 10 medium, 6 low). Critical: path traversal in repor

30. Complete Class Inventory

Class	File	Type	Key Methods
ScanEvent	engine.py	dataclass	type, module_id, target, finding, findings_count, duration, result, modules
EventCollector	engine.py	service	__call__, findings, by_module, timeline
ScanEngine	engine.py	engine	_emit, _resolve_remote_modules, _resolve_local_modules, _build_result, _run
ReconProResult	scanner.py	dataclass	target, findings, score, grade, module_results, elapsed, to_dict
RateLimiter	http_layer.py	service	__init__, acquire
Finding	http_layer.py	dataclass	title, severity, category, module, description, evidence, asset, points_dedupe
ExecutiveSummaryGenerator	report_writer.py	generator	generate, _generate_with_llm, _generate_from_templates, _template_executive,
MarkdownReportBuilder	report_writer.py	builder	build_report
HookManager	plugins.py	manager	register, unregister, fire, clear, list_hooks, save_hooks, load_hooks
PluginInterface	interfaces.py	ABC	abstract: run, name, description
ScanContext	context.py	dataclass	__post_init__, to_args, elapsed_ms
FindingStore	memory.py	store	add, query, recent, stats, to_dicts
AgentBlackboard	memory.py	store	set, get, delete, list_keys, clear, to_dict
CredentialVault	memory.py	store	store, retrieve, delete, list, clear, to_dicts
UnifiedMemoryStore	memory.py	store	findings, graph, blackboard, vault, save, load, clear, stats, to_dict
AsyncSession	async_http.py	wrapper	wraps aiohttp.ClientSession with cookie jars, redirect recording
AdaptiveLimiter	async_http.py	limiter	token-bucket with EMA tracking, hostname cache
ConnectionPool	connection_pool.py	pool	_get_ssl_context, _build_headers, probe, _record_stats, stats, reset_stats,
Proxy	proxy.py	dataclass	to_dict, from_dict
RotationStrategy	proxy.py	enum	ROUND_ROBIN, RANDOM, LEAST_CONNECTIONS, GEO_DISTRIBUTED
ProxyPool	proxy.py	manager	load_from_file, load_from_url, add_proxy, health_check, get_proxy, rotate, s
TorProxyManager	proxy.py	manager	check_tor_available, new_circuit, get_tor_session, verify_new_ip
SynScanner	raw_sockets.py	scanner	scan, _scan_scapy, _scan_connect
UdpProber	raw_sockets.py	prober	probe
TtlAnalyzer	raw_sockets.py	analyzer	analyze, _estimate_initial_ttl
ArpSpy	raw_sockets.py	spy	discover, detect_spoofing
SecretClassifier	entropy.py	classifier	classify
ContextAnalyzer	entropy.py	analyzer	analyze_file, analyze_string
GeoIPLookup	geoip.py	lookup	enrich_ip, enrich_ips, save, clear_cache, get_cache_stats
BatchGeoIP	geoip.py	batch	enrich_batch, get_stats
SubdomainResult	subdomains.py	dataclass	domain, subdomains, resolved, sources_used, total_unique_ips, elapsed
SubdomainEnumerator	subdomains.py	enumerator	enumerate, 11 source methods, _resolve_subdomains
JsEndpointExtractor	api_discovery.py	extractor	extract, _match_patterns
GraphQLIntrospector	api_discovery.py	introspector	probe_introspection, _run_query

Class	File	Type	Key Methods
OpenAPISpecParser	api_discovery.py	parser	parse_spec, _fetch_spec
LinkCrawler	api_discovery.py	crawler	crawl, _extract_links, _extract_forms
AuthMatrix	api_discovery.py	tester	test_endpoints, _test_endpoint
GrpcIntrospector	api_discovery.py	introspector	detect_grpc, probe_grpc
TunnelType	tunnel_detect.py	enum	DNS/ICMP/IPv6/GRE/SSH/HTTP/HTTPS/TCP_OVER_UDP
TunnelDetector	tunnel_detect.py	detector	detect_all, 8 tunnel check methods
ExfilChannelMapper	exfil_channels.py	mapper	map_channels, 9 channel check methods
ExfilRiskScorer	exfil_channels.py	scorer	score_channels, generate_report
ShadowITScanner	shadow_it.py	scanner	scan, 7 check methods, classify_asset, generate_report
WebExtractor	supply_chain.py	extractor	extract, _parse_html, 4 extract methods
GitHubScraper	supply_chain.py	scraper	scrape_repo, _fetch_github_api, _parse_repo_data
SupplyChainAnalyzer	supply_chain.py	analyzer	analyze, 5 check methods
DriftSnapshot	drift_monitor.py	dataclass	10 snapshot fields
InfrastructureDriftMonitor	drift_monitor.py	monitor	capture_snapshot, load_snapshots, compare, 8 diff methods
DriftRiskScorer	drift_monitor.py	scorer	score_event, score_events, generate_report
NetworkDiscovery	netmap.py	discovery	discover_hosts, ping_sweep, arp_discover, mdns_discover, port_scan_hosts
TrustMapper	netmap.py	mapper	build_trust_graph, find_ssh_chains, find_docker_topology, find_lateral_paths
SegmentationChecker	netmap.py	checker	check_zones, check_cross_zone, check_internet_access, check_dns_exfil
SecurityKnowledgeGraph	knowledge_graph.py	graph	add_node, add_edge, remove_node, get_node, get_neighbors, shortest_path, bl
Entity	social_graph.py	dataclass	entity_id, entity_type, name, properties
Relationship	social_graph.py	dataclass	source, target, rel_type, weight, properties
SocialGraph	social_graph.py	graph	add_entity, add_relationship, get_entity, shortest_path, centrality, commun
GraphIntelligence	social_graph.py	analyzer	build_graph, analyze, export_graphviz
AttackGraph	attack_graph.py	graph	add_node, add_edge, build_from_findings, find_attack_paths, find_lateral_mov
OSSignature	quantum_fingerprint.py	dataclass	17 fields for OS fingerprint matching
TimingSample	quantum_fingerprint.py	dataclass	label, elapsed_ms, payload_size, status_code, response_bytes, error
SubProbeResult	quantum_fingerprint.py	dataclass	probe_name, samples, derived_value, description, confidence + properties
ChainHunter	chain_engine.py	hunter	Complex chain analysis with many methods
KillChainAnalyzer	kill_chain.py	analyzer	MITRE ATT&CK kill chain mapping engine
AttributionEngine	attribution.py	engine	TTP-based nation-state attribution
DefenseGenerator	defense.py	generator	generate_code, _generate_flask, _generate_waf, _generate_headers, _generate
CognitiveSecurityAnalyzer	cognitive_sec.py	analyzer	Bias detection, manipulation analysis
FindingClassifier	ai_analyst.py	classifier	classify_finding (24+ categories)
FindingCorrelator	ai_analyst.py	correlator	dedup, grouping, attack chain detection
ExploitabilityEstimator	ai_analyst.py	estimator	Effort and exploitability analysis
BusinessImpactAnalyzer	ai_analyst.py	analyzer	Production vs dev impact assessment
AttackPathDetector	ai_analyst.py	detector	Multi-step attack path discovery

Class	File	Type	Key Methods
RemediationPrioritizer	ai_analyst.py	prioritizer	Risk-based remediation ordering
AIAnalystEngine	ai_analyst.py	engine	Full AI analyst pipeline composition
ExtendedFinding	ai_analyst.py	dataclass	Finding + MITRE/CWE/CAPEC/CVSS/remediation
AIRedTeam	ai_red_team.py	engine	GORGON 15-stage analytical engine
AICVEDB	ai_cve_db.py	database	search, enrich, stats
ThreatIntelCenter	threat_intel.py	center	lookup_cve, lookup_cwe, lookup_capec, lookup_mitre, enrich_finding
ThreatFeedAggregator	threat_feeds.py	aggregator	fetch_feeds, parse_feed, aggregate, get_threats
IntelligencePipeline	intelligence_pipeline.py	pipeline	analyze (composes AI analyst + attack graph + threat intel)
ComplianceMapper	compliance.py	mapper	map_findings, generate_report, _map_iso27001, _map_nist, _map_pci, _map_cis
PassiveIntelCollector	passive_intel.py	collector	collect_dns, collect_whois, collect_headers, collect_all
RatingsEngine	ratings.py	engine	rate_finding, rate_target, compute_dread, get_risk_level
CVERadar	cve_radar.py	radar	enrich, lookup, _fetch_nvd, _parse_cve
CrossValidator	cross_validator.py	validator	validate, deduplicate, correlate, confidence_score
BenchmarkTracker	benchmark.py	tracker	record, get_history, get_trend, compare, generate_report
ObservabilityManager	observability.py	manager	log, metric, health_check, get_status, get_metrics
ScanProfiler	profiler.py	profiler	start, stop, record, get_profile, generate_report
DiagnosticRunner	diagnostics.py	runner	run_all, check_python, check_os, check_network, check_permissions, generate
SecurityManager	security.py	manager	sanitize_input, validate_token, generate_token, check_permissions
EvasionEngine	evasion.py	engine	encode_payload, fragment_request, bypass_waf, generate_variants
MITMAnalyzer	mitm.py	analyzer	check_cert_pinning, check_hsts, check_tls_config, analyze
Fuzzer	fuzzer.py	fuzzer	fuzz_url, fuzz_params, fuzz_headers, fuzz_paths, run
ToolManager	tool_sdk.py	manager	register_tool, run_tool, parse_output, list_tools
WebhookManager	webhooks.py	manager	register, unregister, fire, process, list_webhooks
CollabManager	collab.py	manager	share_scan, list_shared, get_shared, delete_shared
SwarmAgent	swarm.py	agent	run, _scout_phase, _hacker_phase, _coder_phase, _guardian_phase
AdversarialEngine	adversarial.py	engine	run, _hacker_phase, _coder_phase, _verify_phase
WishEngine	wishes.py	engine	parse, fulfill, 7 wish handlers
ChatSession	chat.py	session	process, _parse_command, 8 handlers
NexusApp	nexus_tui.py	tui	compose, on_mount, 20+ reactive methods
HelpOverlay	nexus_help.py	overlay	compose, on_button_pressed, on_key
GraphRenderer	graph_ui.py	renderer	render_tree, render_digraph, render_table, render_ascii_graph
ASTAnalyzer	ast_analyzer.py	analyzer	analyze_python, analyze_javascript, analyze_typescript, detect_patterns, gen
IaCAuditor	iac_audit.py	auditor	audit_terraform, audit_cloudformation, audit_dockerfile, audit_kubernetes, g
ContainerSecurityAnalyzer	container_sec.py	analyzer	analyze_dockerfile, analyze_k8s, analyze_compose
CloudReconEngine	cloud_recon.py	engine	detect_aws, detect_azure, detect_gcp, scan_metadata

31. Complete Function Inventory (Key Functions by Module)

This section catalogs all major standalone functions across the codebase, organized by module. Functions listed below are public API entry points; private helper functions (prefixed with `_`) are documented in their respective module sections above.

Module	Public Functions	Count
utils.py	extract_host, normalize_base_url, validate_target, count_severities, sort_findings_by_severity, severity_to_cvss	17
registry.py	build_module_registry, build_local_modules, get_module_runner, is_local_module, is_remote_module, get_module_info	9
engine.py	concurrent_scan	1
scanner.py	scan, audit_scan	2
parallel.py	blitz_scan	1
async_http.py	async_probe, probe_sync, batch_probe	3
http_layer.py	http_probe, compute_grade, badge_markdown	3
proxy.py	proxy_probe, load_default_pool	2
geoip.py	is_hosting_ip, is_proxy_ip, format_geoip_summary, detect_cloud_provider	4
subdomains.py	discover_ctlogs, discover_dns, discover_subdomains	3
api_discovery.py	discover_api	1
tunnel_detect.py	(class methods only)	0
exfil_channels.py	run_exfil_channels	1
shadow_it.py	run_shadow_it	1
supply_chain.py	run_supply_chain	1
drift_monitor.py	run_drift_monitor	1
netmap.py	run_local	1
entropy.py	shannon_entropy, scan_string, scan_file, scan_directory	4
security.py	run_security_check, sanitize_target, sanitize_path, sanitize_filename, sanitize_html, sanitize_ip, sanitize_email, sanitize_phone, sanitize_url, sanitize_command	13
sanitize.py	sanitize_input, sanitize_path, sanitize_command	3
report_writer.py	generate_narrative, generate_report_md	2
formats.py	export_sarif, export_markdown, export_json, export_html, export_pdf, export	6
server.py	generate_api_token, validate_api_token, revoke_api_token, list_api_tokens, start_server, run_server	7
plugins.py	_ensure_plugin_dir, discover_plugins, run_plugin, create_plugin_template, register_plugin, get_registered_plugins	7
memory.py	(class methods only)	0
history.py	save_scan, list_scans, get_latest, get_scan, diff_scans, clear_history	6
scheduler.py	schedule_scan, list_schedules, cancel_schedule	3
wishes.py	run_wishes	1
chat.py	start_chat	1
webhooks.py	run_webhooks	1
collab.py	run_collab	1

Module	Public Functions	Count
swarm.py	run_swarm	1
adversarial.py	run_adversarial	1
agent.py	run_agent	1
browser_mod.py	take_screenshot	1
fuzzer.py	run_fuzzer	1
diagnostics.py	run_diagnostics	1
profiler.py	run_profiler	1
observability.py	run_observability	1
telemetry.py	telemetry_enabled, send_telemetry, disable_telemetry	3
ai_analyst.py	run_ai_analyst	1
ai_red_team.py	run_ai_red_team	1
ai_cve_db.py	run_ai_cve_db	1
threat_intel.py	run_threat_intel	1
threat_feeds.py	run_threat_feeds	1
intelligence_pipeline.py	(class methods only)	0
attack_graph.py	(class methods only)	0
kill_chain.py	run_kill_chain	1
attribution.py	run_attribution	1
defense.py	run_defense	1
cognitive_sec.py	run_cognitive_sec	1
compliance.py	run_compliance	1
ratings.py	run_ratings	1
cve_radar.py	run_cve_radar	1
cross_validator.py	run_cross_validator	1
benchmark.py	run_benchmark	1
graph_ui.py	render_graph	1

32. Final Statistics & Architecture Map

Grand Totals

Metric	Value
Total Python Source Files	96
Total Test Files	31
Total Deployment/Docs/Script Files	29
Total Files in Project	156
Total Lines of Code (Source)	~110,000
Total Lines of Code (Tests)	~12,627
Total Lines of Code (All)	~121,000+
Total Classes	85+
Total Standalone Functions	200+
Total CLI Commands	28
Total REST API Endpoints	20
Total Scanning Modules	26 (23 remote + 3 local)
Total Advanced Modules	16
Total Intelligence Systems	7 (AI Analyst, Attack Graph, Threat Intel, Pipeline, KG, Social Graph, Feeds)
Total Integrations	6 (Slack, Jira, GitHub, Splunk, PagerDuty, ZAI Stream)
Total TUI Widgets	7
Total Plugin Hooks	9
Total Report Formats	5+ (SARIF, JSON, Markdown, HTML, PDF, AI Markdown)
Total Framework Mappings	13 (MITRE ATT&CK, CWE, CAPEC, CVSS, DREAD, OWASP, ISO 27001, NIST 800-53, SOC2,
Total Algorithms/Detection Engines	21
Total Data Models (Dataclasses)	20+
Total Protocols (Protocol Classes)	5
Total Type Aliases	6
Total Configuration Constants	40+
Total UI Themes	6 (Recon, Matrix, Cyber, Dark, Light, Ocean)
Total Documentation Files	19 (14 docs + 5 ADRs)
Total Test Functions	1,449
Total Test Classes	291
Total Deployment Targets	3 (Docker, Docker Compose, Kubernetes)
Total Scripts	4
Total Engineering Reports	3
Third-Party Dependencies (Required)	0 (pure Python stdlib)
Third-Party Dependencies (Optional)	8 (rich, textual, aiohttp, openai, anthropic, networkx, scapy, playwright)

Architecture Map

ReconPro v11.0 follows a layered architecture pattern with clear separation of concerns. The foundation layer (constants, utils, interfaces) provides shared data types and pure functions. The network layer (http_layer, connection_pool, async_http, proxy, raw_sockets) abstracts all network operations. The orchestration layer (engine, scanner, registry, parallel) manages module execution. The module layer (26 scanning modules in modules/) performs all reconnaissance and analysis. The intelligence layer (ai_analyst, attack_graph, threat_intel, knowledge_graph, social_graph) provides high-level analysis and correlation. The reporting layer (report_writer, reports, formats) generates outputs in 5+ formats. The presentation layer (cli, nexus_tui, server) provides three interfaces (CLI/TUI/REST). Cross-cutting concerns (observability, security, plugins, memory, integrations) span all layers. Deployment is containerized via Docker with Kubernetes orchestration.

Dependency Graph (Internal)

```
constants.py <- utils.py <- http_layer.py <- [all modules]
interfaces.py <- registry.py <- engine.py <- cli.py
memory.py <- knowledge_graph.py <- intelligence_pipeline.py
theme.py <- nexus_tui.py, cli.py, reports.py
plugins.py <- engine.py (hook integration)
security.py <- server.py, sanitize.py, all entry points
observability.py <- engine.py, all modules (structured logging)
formats.py <- cli.py, server.py (export functionality)
```